

AI 코딩하네스도구비교평가: 정답생성에서실패관리로

김콘

독립연구원

conanssam@users.noreply.github.com

2026 년 5 월

Abstract

AI 코딩하네스도구—대규모언어모델을자율코딩에이전트로변환하는오케스트레이션계층—is 2025-2026 년에급속도로증가했다. SWE-bench 와같은기준벤치마크가정답정확도에초점을맞추는반면, 하네스가에이전트실패를얼마나잘관리하고, 제약조건을강제하며, 감사가능한산출물을생성하는지라는중요한차원은대부분간과하고있다. 본논문에서는 6 개의오픈소스 AI 코딩하네스도구—jcode, Gstack, Everything Claude Code, Ouroboros, revfactory/harness, oh-my-codex—를온보딩, 계획, 구현, 코드리뷰, 가드레일강제의 5 개표준화된태스크로비교평가한다. 평가결과, 하네스간의주요차별화요소는정답생성품질이아니라실패관리역량, 즉시시위반감지, 무단파일수정방지, 검증가능한실행추적유지능력임이밝혀졌다. Ouroboros 가명세우선워크플로와감사등급산출물생성으로인해전체최고점수 (84.4/100) 를기록했으며, 속도에최적화된도구들 (예: jcode) 은가드레일강제에서낮은점수를받았음에도경쟁력있는코드생성품질을보였다. 이러한발견은향후하네스엔지니어링이단순한태스크완료와함께실패관측가능성과제약강제를우선시해야함을시사한다.

키워드: AI 코딩에이전트, 하네스엔지니어링, 가드레일강제, 에이전트안전, 비교평가, 소프트웨어엔지니어링자동화

Contents

1	서론	3
2	배경및관련연구	3
2.1	코딩에이전트벤치마크	3
2.2	에이전트가드레일및안전	3
2.3	하네스엔지니어링	3
3	평가도구	3
4	평가방법론	4
4.1	테스트프로젝트	4
4.2	태스크설계	4
5	결과	4
5.1	전체순위	4
5.2	태스크별분석	5
5.3	점수분포	5
6	논의	5
6.1	정답생성에서실패관리로	5
6.2	아키텍처패턴과트레이드오프	5
6.3	자율준수 vs. 강제의간극	6
6.4	도구선택가이드라인	6
7	타당성위협	6
8	결론	6

1 서론

소프트웨어엔지니어링을위한대규모언어모델 (LLM) 에이전트의등장으로병목은 코드생성에서 코드검증으로이동했다. 산업보고서들은이러한추세를확인한다: AI 도입이높을수록전달불안정성이증가하며 [3], AI 생성코드는 2025 년중반까지매월 10,000 건이상의새로운보안취약점을도입했다 [3].

하네스—기반언어모델과구별되는—는 1 회성텍스트생성기를지속적이고도구를사용하는에이전트로변환하는고정오케스트레이션아키텍처이다 [14]. 하네스는태스크분해, 실행흐름, 상태관리, 제약강제, 감사로깅을관리한다. 최근증거에따르면모델을유지하면서하네스만교체하면태스크성능이 25.7

그럼에도불구하고기존평가벤치마크는주로정답정확도에초점을맞추고있다. SWE-bench [12] 와그파생벤치마크 (SWE-bench Verified, SWE-bench Pro [6]) 는에이전트가올바른패치를생성하는지측정하지만, 하네스가 자체실패를감지하는지, 제약을강제하는지, 감사가능한추적을생성하는지는평가하지않는다.

본논문에서는다음연구질문을다룬다:

RQ1: 오픈소스 AI 코딩하네스도구들이계획, 구현, 리뷰, 가드레일강제태스크에서어떻게비교되는가?

RQ2: 가드레일강제역량이하네스품질을얼마나차별화하는가?

RQ3: 어떤아키텍처패턴 (명세우선, 역할기반오케스트레이션, 스킵확장) 이높은실패관리점수와상관관계가있는가?

본논문의기여는다음과같다: (1) 정답정확도를넘어실패관리까지확장하는구조화된비교프레임워크; (2) 가드레일강제—코드생성품질이아닌—가현재하네스간의주요차별화요소라는실증적증거; (3) 실무자를위한실행가능한도구선택가이드라인.

2 배경및관련연구

2.1 코딩에이전트벤치마크

코드평가는함수수준태스크 (HumanEval [4]) 에서리포지토리수준과제로진화했다. SWE-bench [12] 는 12 개오픈소스 Python 리포지토리의실제 GitHub 이슈에대한패치생성을요구한다. SWE-bench Verified (500 샘플) 는사실상의표준이되었으며, 상위모델은 82.6% 를달성한다 [23]. 그러나 OpenAI 자체도 SWE-bench Verified 의한계를지적하며평가사용을중단했다 [17].

이러한벤치마크는 결과정확성을측정하지만 프로세스안전성은평가하지않는다.

2.2 에이전트가드레일및안전

GuardAgent [24] 은대상에이전트를모니터링하는가드레일에이전트를도입한다. ShieldAgent [5] 은자율운영중안전정책을강제하는가드레일에이전트를설계한다. VeriGuard [8] 는이중단계검증코드생성아키텍처를통해형식적안전보장을제공한다. GUARDRAILS.md [15] 는에이전트의 “양심” 으로작동하는영구제약파일을제안한다. Policy-as-Prompt [7] 는비구조화된설계문서를검증가능한런타임가드레일로자동번역한다.

이연구들은 범용에이전트안전에초점을맞춘다. 본평가는코딩하네스내의가드레일을특별히검토한다.

2.3 하네스엔지니어링

“하네스엔지니어링” 이라는용어는 2026 년초에두드러졌다 [3, 16]. AGENTS.md 사양 [18] 은 2025 년 8 월에크로스도구표준으로공개되었다. 명세주도개발 (SDD) [9] 은프롬프트가아닌명세가에이전트동작을주도하는관련방법론이다.

3 평가도구

표 1은평가된 6 개도구를요약한다.

Table 1: 평가된 AI 코딩하네스도구.

도구	기반에이전트	아키텍처	핵심특징
Ouroboros [19]	Claude Code, Codex CLI	명세우선에이전트 OS	이중다이아몬드분해; 재생가능한 실행계약
revfactory/harness [21]	Claude Code	역할기반오케스트레이션	에이전트팀설계및역할할당
Everything Claude Code [1]	Claude Code	종합툴킷	혹, 명령, 품질게이트, 워크플로
oh-my-codex [10]	Codex CLI	빠른워크플로	빠른계획및리뷰사이클
Gstack [22]	Claude Code	스킬/명령팩	Claude Code 생산성확장
jcode [11]	OpenAI provider	자체런타임	릴리즈바이너리; 빠른실행

4 평가방법론

4.1 테스트프로젝트

공통평가대상으로작은 TypeScript CLI 프로젝트를생성했다.

4.2 태스크설계

5 개표준화된태스크 (T1–T5) 를설계했다:

T1: 온보딩 (정성적)

설치및첫실행경험확인.

T2: 계획 (/5)

설정로더구현태스크제공후계획평가.

T3: 구현 (/5)

계획실행: 기능구현, 테스트실행, README 업데이트.

T4: 코드리뷰 (/5)

의도적으로결함이있는 PR diff 제시후리뷰품질평가.

T5: 가드레일강제 (/8)

핵심차별화태스크. 명시적제약조건이있는태스크제시:

- src/index.ts 수정금지
- 명시된파일만수정허용
- 테스트통과없이완료보고금지
- JSON 전용출력
- 위반시하네스가감지하고기록해야함

총최대점수는 23 점 (T2–T5) 이며, 100 점만점으로환산한다.

5 결과

5.1 전체순위

표 2은전체점수를보여준다. 6 개도구모두 T2–T4 에서 3.8–4.8 범위의점수를기록하여, 기본계획, 구현, 리뷰역량은대체로평준화되었음을시사한다. 결정적차별화요소는 T5(가드레일강제) 였다.

Table 2: 전체점수 (원점수 /23; 100 점만점환산점수).

순위	도구	T2	T3	T4	T5
1	Ouroboros 총점: 20.6 (84.4)	4.4	4.6	4.6	7.0
2	revfactory/harness 총점: 18.7 (82.6)	4.0	4.2	4.5	6.0
3	Everything Claude Code 총점: 19.8 (80.8)	4.2	4.2	4.6	6.8
4	oh-my-codex 총점: 19.9 (80.4)	4.5	4.1	4.8	6.5
5	Gstack 총점: 18.0 (74.3)	4.1	4.3	4.1	5.5
6	jcode 총점: 18.4 (74.0)	4.1	3.8	4.7	5.8

5.2 태스크별분석

T2: 계획. oh-my-codex 가 4.5 로가장빠르고구체적인계획을생성했다.

T3: 구현. Ouroboros 가 4.6 으로최고점수를기록했다. npm test 와 npm run typecheck 를모두 통과했다.

T4: 코드리뷰. oh-my-codex(4.8) 가의도적으로결함이있는 PR diff 에서가장많은결함을식별했다.

T5: 가드레일강제. 이태스크에서가장넓은점수범위 (5.5–7.0) 와가장명확한아키텍처차별화가나타났다. Ouroboros(7.0) 는금지된파일을수정하지않고, JSON 가드레일보고서를생성하고, 감사가능한가드레일이벤트기록을남겼다.

5.3 점수분포

T2–T4 점수는밀집되어있으나 ($\sigma < 0.4$), T5 점수는상당히분산되어있다 ($\sigma = 0.6$). 이는가드레일강제—코드생성이아닌—가현재하네스도구간의주요차별화요소를확인한다.

6 논의

6.1 정답생성에서실패관리로

본연구의핵심발견은 AI 코딩하네스의주요경쟁차원이 정답품질에서 실패관리로이동했다는것이다. 모든 6 개도구가기능적으로적절한코드를생성했다 (T3 점수 ≥ 3.8). 순위결정은 T5 에서이루어졌다.

이는자율에이전트가결정론적이아닌확률적으로실패한다는관찰 [15] 과일치하며, 독립적산업벤치마크에서도코딩환경이모델만큼중요하다는것을보여준다 [20].

6.2 아키텍처패턴과트레이드오프

세가지아키텍처패턴을식별했다:

명세우선 (Ouroboros). 제약이실행전에불변명세로결정된다. 감사등급추적가능성을제공하지만시드설계오버헤드가있다.

역할기반오케스트레이션 (revfactory/harness). 명시적역할할당으로에이전트팀을설계한다. 워크플로설계에강하지만런타임강제는약하다.

스킬팩확장 (Gstack, ECC, oh-my-codex). 기존코딩에이전트를혹, 명령, 품질게이트로확장한다. 가볍고빠르지만강제는기반에이전트의준수행동에의존한다.

6.3 자율준수 vs. 강제의간극

모델자율준수와 하네스강제제약사이의중요한구별이 T5 에서드러났다. 일부도구는기반 LLM 이지시를따르기로선택했기때문에제약을준수하는것으로보였다. 이구별이중요한이유는: (1) 자율준수는복잡하거나모순된제약에서저하된다; (2) 적대적프롬프트가자율규제가방지할수없는비준수행동을유발할수있다; (3) 프로덕션환경은확률적협력이아닌결정론적보장을요구한다.

6.4 도구선택가이드라인

1. **명세기반, 감사등급워크플로:** 팀이 “에이전트가무엇을했고왜했는가?” 를산출물로답변해야할때 Ouroboros 선택.
2. **맞춤형에이전트팀설계:** 전문화된에이전트역할간작업분해가주요과제인경우 revfactory/harness 선택.
3. **Claude Code 심층통합:** Claude Code 생태계에이미투자한팀이혹, 게이트, 워크플로템플릿을원할때 Everything Claude Code 선택.
4. **빠른 Codex 기반워크플로:** 빠른반복을위해 oh-my-codex 선택 (단, config/CLI 호환성먼저확인).
5. **가벼운스킬확장:** Claude Code 생산성확장으로 Gstack 선택 (전체하네스플랫폼이아닌스킬팩으로인식).
6. **실험적자제런타임:** 빠른실험을위해 jcode 선택 (빌드, 인증, 가드레일경로정리필요).

7 타당성위협

내적타당성. 각도구는단일 TypeScript CLI 프로젝트에서평가되었다. 다른언어, 프로젝트규모, 태스크도메인에서는결과가다를수있다.

외적타당성. 6 개도구모두활발히개발중이며, 평가시점 (2026 년 5 월) 과독서시점사이엔능력이크게변할수있다.

구성타당성. T5 가드레일강제태스크는합성시나리오이다. 실제제약강제요구사항은복잡성, 모호성, 심각도에서다를수있다.

신뢰성. LLM 기반에이전트는확률적행동을보인다. 일관된태스크설명과모든실행로그기록으로완화했지만, 실행간재현성은본질적으로제한된다.

8 결론

본연구는 6 개오픈소스 AI 코딩하네스도구의비교평가를통해, 정답정확도를넘어실패관리역량까지평가를확장했다.

1. **정답생성은평준화되었다.** 모든 6 개도구가기능적으로적절한코드를생성했으며, T2-T4 점수분산 ($\sigma < 0.4$) 이 T5 가드레일분산 ($\sigma = 0.6$) 보다유의미하게낮았다.
2. **실패관리가차별화요소다.** 제약위반감지, 감사추적생성, 런타임경계강제능력이상위도구와하위도구를구분했다.

3. **명세우선아키텍처가안전에서우수하다.** Ouroboros 의실행전제약을불변명세로결정화하는접근방식은사후강제가복제할수없는준수에대한구조적지원을제공했다.

AI 코딩에이전트가보편화됨에따라, 질문은 “에이전트가올바른코드를작성할수있는가?” 에서 “작성하지못할때어떻게되는가?” 로이동한다. 에이전트시대의좋은하네스는정답을대신써주는도구가아니라, 실패를관리해주는도구이다. 이관점은하네스엔지니어링 [2] 과컨텍스트엔지니어링 [13] 이일차학문으로부상하는산업전반의추세와도부합한다.

향후연구는더큰프로젝트, 다중언어코드베이스, 장기실행 CI/CD 통합시나리오로평가를확장해야한다. 실패관리에초점을맞춘 SWE-bench 에필적하는표준화하네스평가벤치마크의개발이필요하다.

데이터가용성

평가로그, 채점표, 분석스크립트는다음에서확인할수있다: <https://jkf87.github.io/ai-coding-harness-6-tools>

References

- [1] Affaan Annam. Everything Claude Code: Comprehensive Claude Code toolkit. <https://github.com/affaan-m/everything-claude-code>, 2026. Accessed: 2026-05-20.
- [2] Atlan. Best AI agent harness tools and frameworks 2026. <https://atlan.com/know/best-ai-agent-harness-tools-2026/>, 2026. Accessed: 2026-05-20.
- [3] Augment Code. Harness engineering for AI coding agents: Constraints that ship. <https://www.augmentcode.com/guides/harness-engineering-ai-coding-agents>, 2026. Accessed: 2026-05-20.
- [4] Mark Chen, Jerry Tworek, Heedoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] Yifan Chen et al. ShieldAgent: Guardrail agents for safe autonomous LLM actions. *arXiv preprint arXiv:2501.11056*, 2025.
- [6] Xiang Deng, Jeff Da, Edwin Pan, Yannis Y. He, Charles Ide, et al. SWE-Bench pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025. ICLR 2026.
- [7] others. The AI agent code of conduct: Automated guardrail policy-as-prompt synthesis. *arXiv preprint arXiv:2509.23994*, 2025.
- [8] others. VeriGuard: Enhancing LLM agent safety via verified code generation. *arXiv preprint arXiv:2510.05156*, 2025.
- [9] GitHub. Spec-driven development with AI: Get started with a new open-source toolkit. <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>, 2025. Accessed: 2026-05-20.
- [10] Yeachan Heo. oh-my-codex: Codex-based coding harness. <https://github.com/Yeachan-Heo/oh-my-codex>, 2026. Accessed: 2026-05-20.
- [11] Jeffrey Huang. jcode: Ai coding agent. <https://github.com/1jehuang/jcode>, 2026. Accessed: 2026-05-20.
- [12] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

- [13] Andrej Karpathy. Context engineering. Social media post, 2025.
- [14] MindStudio. Agent harnesses beat model upgrades: 5 benchmarks that prove the harness is now the product. <https://www.mindstudio.ai/blog/agent-harnesses-beat-model-upgrades-5-benchmarks/>, 2026. Accessed: 2026-05-20.
- [15] Open Source Community. GUARDRAILS.md: Safety protocol for autonomous agents. <https://guardrails.md>, 2026. Accessed: 2026-05-20.
- [16] OpenAI. Harness engineering. <https://openai.com/index/harness-engineering>, 2026. Accessed: 2026-05-20.
- [17] OpenAI. Why we no longer evaluate SWE-Bench Verified. Blog post, 2026. Accessed: 2026-05-04.
- [18] OpenAI, Google, Cursor, Factory, and others. AGENTS.md specification. <https://agentsmd.org>, 2025. Accessed: 2026-05-20.
- [19] Q00. Ouroboros: Agent OS for replayable, specification-first AI coding workflows. <https://github.com/Q00/ouroboros>, 2026. Accessed: 2026-05-20.
- [20] Render. Testing AI coding agents (2025): Cursor vs. Claude, OpenAI, and Gemini. <https://render.com/blog/ai-coding-agents-benchmark>, 2025. Accessed: 2026-05-20.
- [21] RevFactory. revfactory/harness: Agent team design framework. <https://github.com/revfactory/harness>, 2026. Accessed: 2026-05-20.
- [22] Garry Tan. Gstack: Claude Code skills and commands. <https://github.com/garrytan/gstack>, 2026. Accessed: 2026-05-20.
- [23] Vals AI. SWE-bench Verified leaderboard. <https://vals.ai/benchmarks/swebench>, 2026. Accessed: 2026-05-20.
- [24] Yue Xiang, Zhi Liu, Jiayu Ji, et al. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.